

CPE477/ECG677 – Embedded Security & Machine Learning

Design Assignment 3

DO NOT REMOVE THIS PAGE DURING SUBMISSION:

Name: Chetan Dahal

Group-Partner Name:

Email: dahalc1@unlv.nevada.edu

Github Repository link (root): <https://github.com/autonomize009/autonomize009-Assign>

Youtube Playlist link (root):

<https://www.youtube.com/playlist?list=PLVIuzDVvAe9iHvuCgkgMcRMypPbO1yoZG>

Follow the submission guideline to be awarded points for this Assignment.

Submit the following for all Assignments:

1. In the document, for each task submit the modified or included code (from the base code) with highlights and justifications of the modifications. Also include the comments. If no base code is provided, submit the base code for the first task only.
2. Create a private Github repository with a random name (no CPE477/677, Lastname, Firstname). Place all assignments under the root folder, sub-folder named Assignmentn, with one document and one video link file for each lab, place modified c files named as main.c.
3. If multiple 'c' or 'h' files or other libraries are used, place these files inside the folder.
4. The folder should have a) Word document (see template), b) source code file(s) with other 'c' and 'h' include files, c) text file with youtube video links (see template).
5. Submit the pdf file in canvas before the due date. The root folder of the github assignment directory should have the documentation and the text file with youtube video links.
6. Organize your youtube videos as playlist under the name "EMBSEC&ML". The playlist should have the video sequence arranged as submission or due dates.
7. Only submit pdf documents. Do not forget to upload this document in the github repository and in the canvas submission portal.

Assignment Objectives

This work is focused on learning the MLC features of MEMS-Studio. Data were collected using LSM6DSV16X IMU of X-NUCLEO-IKS4A1 sensor board mounted on STM32 Nucleo-F401RE microcontroller unit. The procedure for collecting & labelling data in MEMS-Studio along with the way of training a decision-tree classifier in MEMS Studio was explored. The capability of real-time streaming of sensor data and real-time classification using MLC on STM32 board was understood and the results were compared with the Random Forest and SVM classifier programmed using Python libraries.

Problem Formulation (Dataset choice)

Wrist/Forearm injuries due to repetitive motion tasks, termed as Repetitive Strain Injury (RSI), is prevalent mainly in industrial or mechanical workshop settings where assembly-line workers or mechanics are subjected to monotonous tasks. They are required to handle the repetitive tasks with their wrists and forearms holding different tools/machines. So, they are susceptible to RSI while performing repetitive tasks like screwdriving, hammering, filing, snipping, etc. using the bench tools, especially if the position of their wrists and forearms are inappropriate during the tasks.

Hence, this work utilizes the STM32 board and MEMS Studio to demonstrate the use of a MCU for real-time detection of type of operation being performed by a worker, along with the safe or unsafe working posture of wrist/forearms. Specifically, it focuses on the two operations: screwdriving and snipping/cutting, where the wrist position needs to be straight with the forearm for safe operation.

Data Collection Procedure

As stated, the data is collected with X-NUCLEO-IKS4A1 sensor board mounted on STM32 Nucleo-F401RE microcontroller unit. The whole procedure for data collection is described below:

1. DataLogExtended firmware was loaded into the STM32 microcontroller unit for real-time streaming of sensor data.
2. The LSM6DSV16X sensor was selected for both accelerometer and gyroscope data. Using MEMS Studio, the full scale of accelerometer and gyroscope were set to 8g and 1000dps respectively with output data rate of 30 Hz was set for both sensors.
3. The MCU-sensor board was tied to the back of the palm for data collection while the board was connected to the laptop via a USB cable.
4. The screwdriving operation was imitated while holding a screwdriver in the same hand with the sensor board. The motion of hand, where wrist is in-line with the forearm while screwing (rotation given by forearm), was considered as a correct operation while the motion of screwing with the wrist in bent position was considered as incorrect operation as that has risk of wrist pain and injury. The same condition was used for snipping. Snipping operation was mimicked using a scissors.
5. Data were collected first for screwing operation. The hand was kept in rest for a certain time and then correct screwing operation was performed for almost a minute followed

by operation with incorrect posture. The speed of operation and rotation of motion were also varied to generalize the data. After saving the data for screwing operation, snipping was mimicked using scissors. Data were obtained for stationary hand position, incorrect operation with bent wrist while snipping, followed by correct operation. The process was repeated to obtain several data for about 3-5 minutes.

Data Labeling

Data analysis feature of MEMS Studio was utilized to label the data for different operations. Each data files were loaded for analysis and the data corresponding to each operation were labeled as shown in Figure 1 (for screwdriving) and Figure 2 (for snipping).

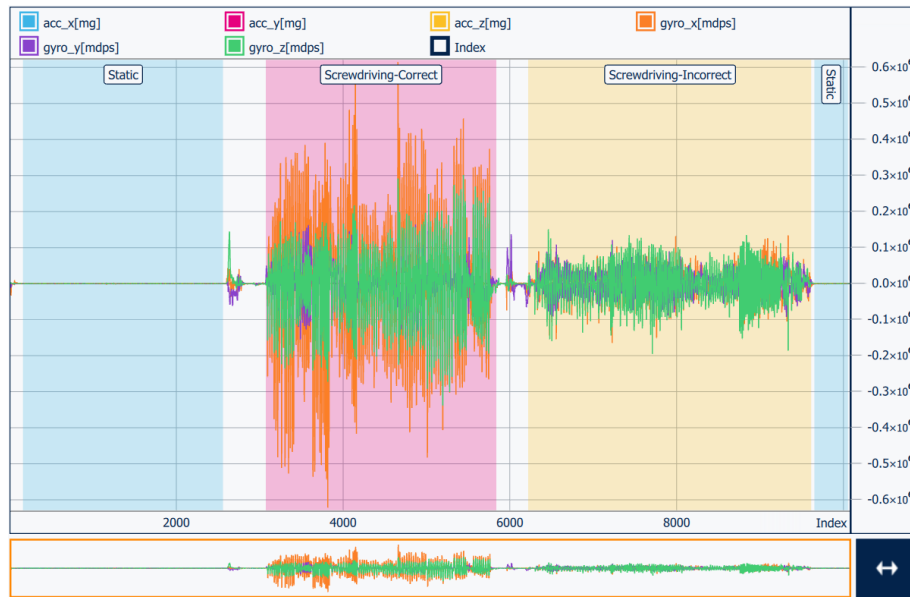


Figure 1: Data Labeling for screwdriving operation

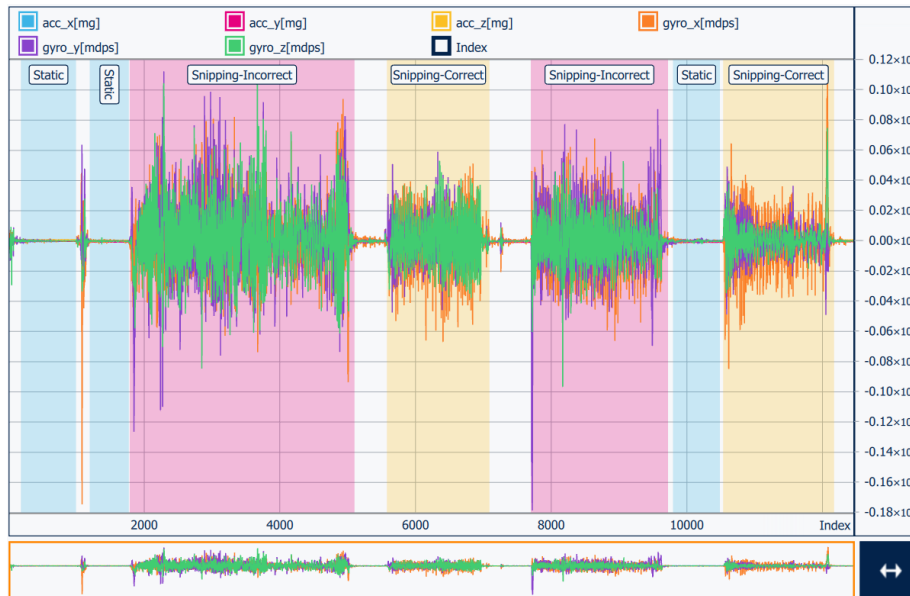


Figure 2: Data Labeling for Snipping operation

The labelled data were saved in separate csv files for each class of data. Total 5 classes of data set were now ready for the training: Static, Screwdriiving-Correct, Screwdriiving-Incorrect, Snipping-Correct and Snipping-Incorrect.

MLC decision-tree training using MEMS Studio

MEMS Studio provides an advance feature where a decision-tree classifier can be trained for real-time data classification. Following steps were implemented for training the MLC via MEMS Studio:

1. Data patterns: The csv files for each data class were loaded and their corresponding class names were entered.
2. AFS tools settings: MEMS Studio provides automatic filters and feature selection tools, where it automatically selects the appropriate window length and features based on the specified settings. The MLC output data rate was set to 30Hz with the input settings of accelerometer and gyroscope sensors as 8g, 1000dps full scale. The automatically selected 15 features were used as input features for training the MLC. Peak-to-Peak, mean, max, min gyroscopic and accelerometer values were some of the selected features.
3. ARFF Generation: With the selected input settings and auto-selected features, ARFF file was generated, which stored the list of selected features and all datasets in the single file. This file can be then used to train ML models for the sensor's MLC. Figure 3 shows the screenshot of a portion of the generated ARFF file.

```
relation 'MLC'

@attribute F1_MINIMUM_GYR_X numeric
@attribute F2_MAXIMUM_GYR_X numeric
@attribute F3_VARIANCE_GYR_X numeric
@attribute F4_PEAK_TO_PEAK_GYR_X numeric
@attribute F5_ABS_MEAN_GYR_V numeric
@attribute F6_ENERGY_GYR_X numeric
@attribute F7_MAXIMUM_ACC_Y numeric
@attribute F8_MAXIMUM_ACC_X numeric
@attribute F9_MAXIMUM_ACC_Z numeric
@attribute F10_MEAN_ACC_Z numeric
@attribute F11_MEAN_ACC_Y numeric
@attribute F12_ABS_MINIMUM_ACC_V numeric
@attribute F13_ENERGY_ACC_Y numeric
@attribute F14_MINIMUM_ACC_Y numeric
@attribute F15_ENERGY_ACC_X numeric
@attribute class {Static, Screwdriiving-Correct, Screwdriiving-Incorrect, Snipping-Correct, Snipping-Incorrect}

@data
-0.0170898, 0.0164795, 4.42266e-05, 0.0335693, 0.00617599, 0.00358772, -0.479248, -0.362305, 0.793457, 0.785645, -0.486084, 0.994629, 19.0469, -0.490234, 10.7813, Static
-0.012207, 0.00610352, 2.03252e-05, 0.0183105, 0.00511932, 0.00186348, -0.480225, -0.362305, 0.79248, 0.785645, -0.486816, 0.994629, 19.1094, -0.491211, 10.7578, Static
-0.0164795, 0.00793457, 3.1352e-05, 0.0244141, 0.00630188, 0.00266647, -0.478271, -0.360352, 0.793457, 0.786621, -0.484619, 0.994629, 19.0781, -0.493408, 10.75, Static
-0.0134277, 0.00793457, 1.78218e-05, 0.0213623, 0.00567245, 0.00173473, -0.479248, -0.359131, 0.793457, 0.786621, -0.485107, 0.994629, 19.0313, -0.489258, 10.7422, Static
-0.010376, 0.00976563, 1.32918e-05, 0.0201416, 0.00494766, 0.0012064, -0.479248, -0.360352, 0.79541, 0.786621, -0.485352, 0.994141, 19.0469, -0.489258, 10.7344, Static
-0.00793457, 0.00549316, 7.39098e-06, 0.0134277, 0.00380325, 0.000601292, -0.477295, -0.360352, 0.794434, 0.787109, -0.481934, 0.994629, 18.9375, -0.487305, 10.7109, Static
```

Figure 3: Snip of ARFF file

4. Decision tree generation: With the generated ARFF file, the data logs were used to train a decision tree classifier by MEMS Studio. The maximum depth of the tree was set to 15 with min leaf samples of 1. 80% of data were used for training and generating the decision tree.
5. Config generation: Class output values for different class were set as shown in Figure 4.

Metaclassifier [Decision tree 1]

Class output values:

Static	Screwdriiving-Correct	Screwdriiving-Incorrect	Snipping-Correct	Snipping-Incorrect
0	3	6	9	12

Figure 4: Class output values for different classes

The configuration file based on these output values were then generated and loaded into the sensor. At this point, the STM32 board with the IKS4AI sensor is ready for real-time data classification.

6. Decision tree output viewer: The real-time classification of data was visualized using the output viewer of MEMS Studio, where the classification is demonstrated in step-pulse like format as seen in Figure 5. The values at right represents the class output labels as described in Figure 4.

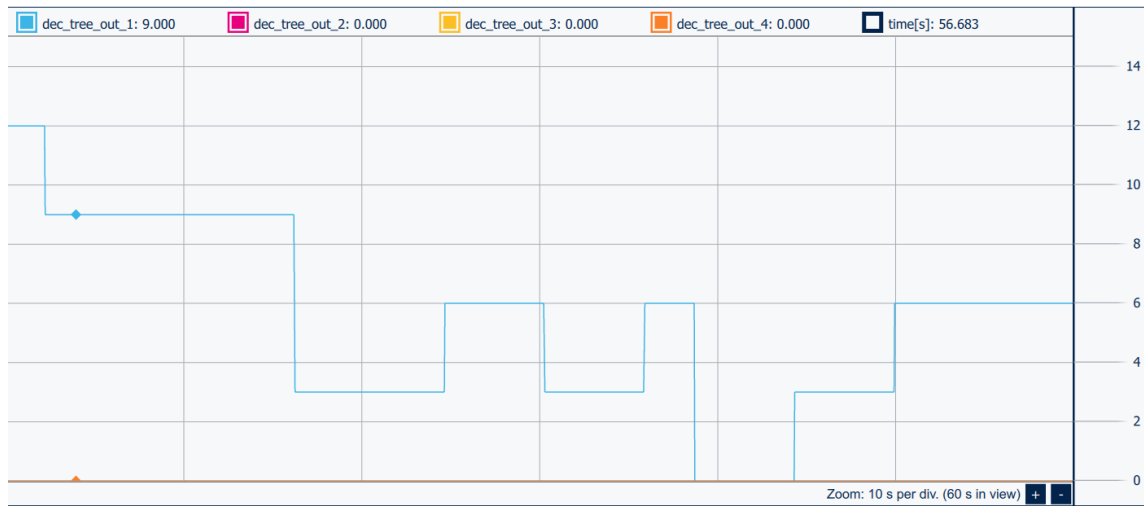


Figure 5: Real-time data classification by trained MLC

Results – Decision Tree MLC

Figure 6 and 7 shows the metrics of decision tree classifier generated by the MEMS Studio. The classification accuracy of 99.31% was achieved. As seen in the confusion matrix, all the data for static, correct and incorrect screwdriving were classified correctly. There was slight misclassification for the cases of correct and incorrect snipping operation class. Both the average precision and recall of the model was 0.99, which indicates almost perfect classification for the given test dataset.

ST_decision_tree_20251104_005208_334.txt					
Confusion Matrix:					
	Static	Screwdriving-Correct	Screwdriving-Incorrect	Snipping-C	
Static	115	0	0	0	
Screwdriving-Correct	0	34	0	0	
Screwdriving-Incorrect	0	0	41	0	
Snipping-Correct	0	0	0	35	1
Snipping-Incorrect	0	0	0	1	64
Total Number of Instances: 291					
Correctly Classified Instances: 289					
Incorrectly Classified Instances: 2					
Accuracy: 99.31 %					
Cohen's kappa: 99.08 %					
Report:	precision	recall	support		
Static	1.00	1.00	115		
Screwdriving-Correct	1.00	1.00	34		
Screwdriving-Incorrect	1.00	1.00	41		
Snipping-Correct	0.97	0.97	36		
Snipping-Incorrect	0.98	0.98	65		
avg/total	0.99	0.99	291		

Figure 6: Metrics for Decision Tree Classifier



Figure 7: Confusion Matrix for decision tree classifier

The actual decision tree generated by the MEMS Studio can be seen in Figure 8, where different conditions on selected input features are checked along the tree to arrive to the output decision.

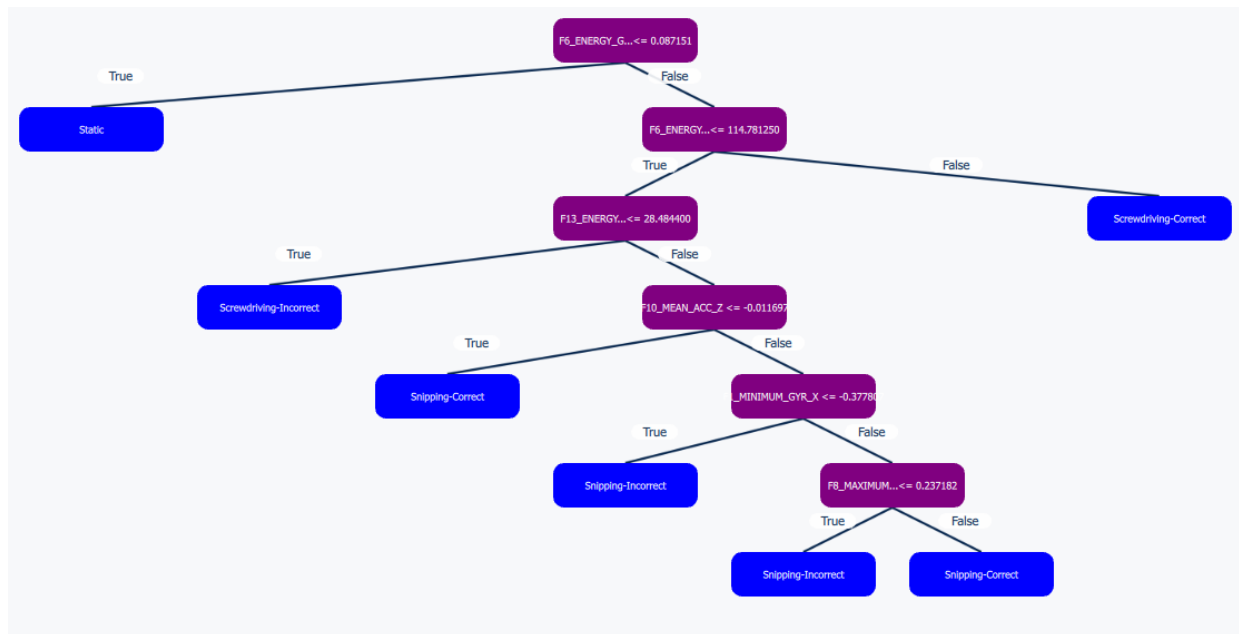


Figure 8: Decision Tree generated by MEMS Studio

SVM and Random Forest Classifier in Python

Python script was written to develop a Random Forest classifier and support vector machine classifier by training the model with the same dataset as the decision tree. The dataset was loaded as a pandas dataframe and input features and output label were separated followed by splitting the input-output data into training and testing dataset with 80% data for training.

K-fold cross validation was setup for both classifiers, where k = 5 was used with comparison based on mean of accuracy was implemented. The “stratified” method keeps the equal proportion of output classes in both training and validation/test dataset.

```
# Setup for cross-validation
cross_valid = StratifiedKFold(n_splits = 5, shuffle = True, random_state = 44)
scoring = "accuracy"
```

Random Forest Classifier Procedure

1. Python library “sklearn.ensemble.RandomForestClassifier” was used for defining the random forest classifier.
2. The classifier was trained and tested for different hyperparameters which were tuned with the cross-validation process. Following parameters were tested:

```
param_grid_rf = {
    "n_estimators": [100, 200, 400],
    "max_depth": [None, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", 0.5], # 0.5 = half of features
    "bootstrap": [True]
}
```

Number of decision tree to use, maximum depth from root to leaf, minimum number of samples required in a node for splitting, min no. of samples allowed in any leaf, max no. of features allowed to be considered at each node were some of the hyperparameters that were tuned using the cross-validation process.

3. Grid search was used on the cross-validation and hyperparameters defined above and the best model was fitted on the training set.

```
grid_rf = GridSearchCV(
    estimator = rf,
    param_grid = param_grid_rf,
    cv = cross_valid,
    scoring = scoring,
    refit = True, # refit best model on the full training set
    n_jobs = -1,
    verbose = 0
)
grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test) # prediction for test set using best
random forest parameters
```

4. The best hyperparameters was then obtained and used for the prediction in the test dataset.

SVM Classification Procedure

Similar procedure was followed for training SVM, but a pipeline was defined first because SVM requires the input features to be standardized for better performance. Each fold during cross validation has different training and validation set, which requires the standardization of dataset during each fold with its corresponding training set. Hence, the pipeline was defined here for preventing any possible errors during standardization which might lead to data leakage. Different values of hyperparameters were also defined for tuning the model as shown below.

```
# SVM pipeline creation to prevent leakage during dataset standardization
svm_pipe = Pipeline([("scaler", StandardScaler()), ("svc", SVC(kernel = "rbf",
probability = False, random_state = 42))])

param_grid_svm = {"svc__C": [0.1, 1, 3, 10, 30, 100], "svc__gamma": ["scale",
0.01, 0.03, 0.1, 0.3, 1.0]}
```

The other process was similar to the case of Random Forest classifier described earlier.

Results – Random Forest and SVM

Figure 9 and 10 shows the classification accuracy, confusion matrix and other metrics for random forest and SVM respectively. The classification accuracy obtained for random forest was 0.9831. Furthermore, the best hyperparameters obtained through cross validation are also shown in the respective figures. The macro average of the model for precision, recall and f1-score were equal to 0.9857, 0.9714 and 0.9722 respectively.

```
----- RANDOM FOREST -----
Best params: {'bootstrap': True, 'max_depth': None, 'max_features': 0.5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Test Accuracy: 0.9831
Classification Report:

```

	precision	recall	f1-score	support
Screwdriving-Correct	1.0000	1.0000	1.0000	7
Screwdriving-Incorrect	1.0000	1.0000	1.0000	8
Snipping-Correct	1.0000	0.8571	0.9231	7
Snipping-Incorrect	0.9286	1.0000	0.9630	13
Static	1.0000	1.0000	1.0000	24
accuracy			0.9831	59
macro avg	0.9857	0.9714	0.9722	59
weighted avg	0.9843	0.9831	0.9827	59

```
Confusion Matrix:
[[ 7  0  0  0  0]
 [ 0  8  0  0  0]
 [ 0  0  6  1  0]
 [ 0  0  0 13  0]
 [ 0  0  0  0 24]]
```

Figure 9: Results of Random Forest

The classification accuracy for SVM was 0.9322 with C value of 3 and gamma of 0.03 as the best values chosen by cross-validation process. The macro average precision, recall and f1-score of the best model was equal to 0.9152, 0.9254 and 0.9167 respectively.

```

----- SVM -----
Best params: {'svc__C': 3, 'svc__gamma': 0.03}
Test Accuracy: 0.9322
Classification Report:

```

	precision	recall	f1-score	support
Screwdriving-Correct	1.0000	1.0000	1.0000	7
Screwdriving-Incorrect	1.0000	1.0000	1.0000	8
Snipping-Correct	0.6667	0.8571	0.7500	7
Snipping-Incorrect	0.9091	0.7692	0.8333	13
Static	1.0000	1.0000	1.0000	24
accuracy			0.9322	59
macro avg	0.9152	0.9253	0.9167	59
weighted avg	0.9404	0.9322	0.9336	59

```

Confusion Matrix:
[[ 7  0  0  0  0]
 [ 0  8  0  0  0]
 [ 0  0  6  1  0]
 [ 0  0  3 10  0]
 [ 0  0  0  0 24]]

```

Figure 10: Results of SVM

Confusion matrix comparisons

Figure 6, 9 and 10 shows the confusion matrix for Decision tree – MLC, Random Forest and SVM. MEMS Studio constructed confusion matrix with the result of whole 291 instances where as RF and SVM used 59 test instances for confusion matrix. There were 2 errors in Decision Tree (DT) between the correct and incorrect snipping operation. For RF, only one instance of correct snipping was misclassified as incorrect snipping while for SVM, there were 4 misclassification and all of them were also between correct and incorrect snipping operations.

Observations and Conclusion

As seen from the confusion matrices, static hand position, correct and incorrect screwdriving operations were accurately classified by all classifiers for every instances. However, all the classifiers showed some confusion for the classification of safe and unsafe snipping operations. Decision Tree showed the highest overall accuracy of 99.31% followed by Random Forest with accuracy of 98.31% and SVM with 93.22% accuracy indicating RF as better than SVM for that specific data split.

The models were accurate for static and screwdriving operations as each of those 3 operations have kinematically different motion of wrist and forearms. Correct and incorrect snipping operations has only subtle posture and motion differences and hence all classifiers showed some misclassification for these operations. With the selected input features, models weren't able to accurately separate the snipping operations, specially by SVM.

The performance can be further improved by selecting more relevant input features, collecting more data with different forces, wrist angle and operation speeds so the model can generalize to different operating conditions for each operation.

Declaration

I understand the Student Academic Misconduct Policy -
<http://studentconduct.unlv.edu/misconduct/policy.html>

“This assignment submission is my own, original work”.
Chetan Dahal